

Looking at the approved structure:

Course Outline: Communicating with External APIs

Title: Communicating with External APIs **Learnpack:** + Comunicarse con APIs externas **Prerequisite:** Skill 13 — Web Architecture and Data Flows **Target Audience:** Beginner developers with frontend experience, first contact with full API communication **Total Lessons:** 24 **Estimated Total Length:** ~12,000 words **Format:** Mixed (25% hands-on = 6 lessons)

Course Description

APIs are the backbone of modern web applications. Every time your frontend loads data, submits a form, or authenticates a user, it is talking to an API. This course teaches you how that conversation works — from the structure of a request to the meaning of a response — so you can build frontends that communicate confidently with any backend.

You will learn the full anatomy of an HTTP request: methods, headers, and bodies. You will learn how to read what the server sends back, including status codes and response data. You will also get hands-on experience using Postman, the industry-standard tool for testing APIs before writing a single line of frontend code.

By the end of this course, you will be able to look at any API endpoint, understand what it expects, send the right request, and handle what comes back — with or without the help of a coding agent.

Course Philosophy

- This course emphasizes:
- **Evidence over assumption:** Always inspect what the API actually returns before writing logic around it
 - **Tools first:** Use Postman to test and understand an API before integrating it into code
 - **Stateless mindset:** Every request must be self-contained — the server remembers nothing
-

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - APIs Fundamentals	3	~1,500
02 - HTTP Methods	4	~2,000
03 - Building Requests	5	~2,500
04 - Reading Responses	3	~1,500
05 - Tools and Authentication	4	~2,000

Section	Lessons	Estimated Words
06 - Assessment	2	~1,400
07 - Conclusion	1	~450
Total	23	~11,800

Section 00: Welcome

00.0 Welcome to the Course (~450 words)

Learning Objectives:

- Understand what this course covers and why it matters
- Know what to expect by the end of the course

Content Outline:

- What problem this course solves: your frontend needs to talk to a backend
- What you will be able to do after this course
- How this course connects to what came before (data flows, promises, fetch) and what comes next (building your own API)
- A quick preview of the tools you will use: browser, Postman, and code

Transitions: This lesson sets the stage. The next section starts from scratch — what an API actually is and how the request-response cycle works.

Key Principles:

- Understanding the full request-response cycle makes you a better frontend developer, even if you never touch the backend
- Postman is not optional — it is how professionals verify API behavior before writing code

Examples:

- When you log into a website, your browser sends a POST request to an API and receives a token back
 - When a page loads a list of products, it sends a GET request and renders whatever the API returns
-

Section 01: APIs Fundamentals

01.0 What Is an API? (~500 words)

Learning Objectives:

- Define what an API is in plain terms
- Explain the role of an API in a web application
- Distinguish between a frontend, a backend, and an API

Content Outline:

- API stands for Application Programming Interface — a contract between two systems
- The frontend asks, the backend answers: client-server model revisited
- APIs are everywhere: weather apps, login systems, payment forms
- REST APIs: the most common type you will encounter
- The URL as an address: what each part of an endpoint URL means

Transitions: Now that you know what an API is, the next lesson explains the mechanics of how a conversation between client and server actually happens.

Key Principles:

- An API is a contract: it defines what you can ask for and what you will get back
- The frontend never talks directly to the database — the API is always in between
- REST APIs use URLs and HTTP methods to define operations

Examples:

- <https://api.weather.com/v1/forecast?city=Madrid> — a URL that returns weather data for Madrid
 - A login form sends credentials to `POST /auth/login` and receives a token if successful
 - A product page calls `GET /products/42` to load details for product number 42
-

01.1 The Request-Response Cycle 📖 (~500 words)

Learning Objectives:

- Describe the full lifecycle of an HTTP request
- Identify the key components of a request and a response
- Understand what happens between the moment you call `fetch()` and the moment data arrives

Content Outline:

- Step by step: client sends request → server processes → server sends response
- Anatomy of a request: method, URL, headers, body
- Anatomy of a response: status code, headers, body
- Synchronous vs asynchronous: why requests don't block the browser
- The role of DNS, TCP, and HTTPS (brief, conceptual — no deep networking)

Transitions: You now understand the shape of a request and a response. The next lesson puts this into practice — you will send your first real API call and inspect what comes back.

Key Principles:

- Every HTTP interaction has exactly two actors: a client that asks and a server that answers
- The request and the response are both structured messages with predictable parts
- The browser does not freeze while waiting — thanks to asynchronous execution

Examples:

- Typing a URL in the browser is a GET request — the server responds with HTML
- A `fetch()` call in JavaScript constructs a request programmatically

- The Network tab in DevTools shows every request and response in real time
-

01.2 First API Call 📖 (~600 words)

Challenge Prompt: Use the `fetch()` API to make a GET request to

`https://jsonplaceholder.typicode.com/users/1`. Log the full response status and the parsed JSON body to the console. Then make a second GET request to

`https://jsonplaceholder.typicode.com/users/999` (a user that does not exist) and log the status of that response too. Do not use `async/await` — use `.then()` chaining for both requests.

Solution Code:

```
fetch("https://jsonplaceholder.typicode.com/users/1")
  .then(response => {
    console.log("Status:", response.status);
    return response.json();
  })
  .then(data => {
    console.log("User data:", data);
  });

fetch("https://jsonplaceholder.typicode.com/users/999")
  .then(response => {
    console.log("Status for missing user:", response.status);
  });
```

Challenge Code:

```
// Challenge: Make two GET requests using fetch() and .then() chaining
// Do NOT use async/await

// Request 1: GET https://jsonplaceholder.typicode.com/users/1
// - Log the response status
// - Log the parsed JSON body

// your code here

// Request 2: GET https://jsonplaceholder.typicode.com/users/999
// - Log the response status only

// your code here
```

Section 02: HTTP Methods

02.0 GET and POST 📖 (~500 words)

Learning Objectives:

- Explain the purpose of GET and POST methods
- Know when to use GET vs POST
- Understand how each method affects the request structure

Content Outline:

- GET: reading data, no body, parameters in the URL (query strings)
- POST: sending new data to the server, body required, not idempotent
- Safe vs unsafe methods: GET does not change server state, POST does
- Common use cases: GET for loading, POST for creating or submitting
- Why you should never send sensitive data in a GET query string

Transitions: GET and POST cover most everyday interactions. The next lesson covers the remaining methods used to update and delete data.

Key Principles:

- GET is for reading — it should never change anything on the server
- POST is for creating — the server receives new data and does something with it
- Sensitive data (passwords, tokens) must never travel in a URL

Examples:

- `GET /products` loads the full product list
 - `GET /products?category=shoes` filters the list by category using a query string
 - `POST /users` with a JSON body creates a new user account
-

02.1 PUT, PATCH and DELETE 📖 (~500 words)

Learning Objectives:

- Distinguish between PUT, PATCH, and DELETE
- Know when each method is appropriate
- Understand idempotency in the context of HTTP methods

Content Outline:

- PUT: replace an entire resource with the data you send
- PATCH: update only the fields you specify, leave the rest unchanged
- DELETE: remove a resource — no body needed, just the URL
- Idempotency: calling PUT or DELETE multiple times produces the same result
- Why PATCH is preferred over PUT for most real-world update operations

Transitions: You now know all five main HTTP methods. The next lesson helps you decide which one to use given a specific operation.

Key Principles:

- PUT replaces everything; PATCH updates only what you send

- DELETE is destructive — always confirm the correct resource URL before sending
- Idempotent methods are safer to retry if a network error occurs

Examples:

- **PUT** `/users/5` with a full user object replaces the entire user record
 - **PATCH** `/users/5` with `{ "email": "new@email.com" }` updates only the email
 - **DELETE** `/posts/12` removes post number 12 permanently
-

02.2 Choosing the Right Method 📖 (~500 words)

Learning Objectives:

- Map a given operation to the correct HTTP method
- Recognize common mistakes in method selection
- Read an API operation description and identify the appropriate method

Content Outline:

- The CRUD-to-method mapping: Create→POST, Read→GET, Update→PUT/PATCH, Delete→DELETE
- How to read API documentation to find the expected method
- Anti-pattern: using POST for everything because it "always works"
- Anti-pattern: using GET to delete or modify data
- Real-world example: walking through a simple task management API

Transitions: With methods covered, the next section moves inside the request — starting with headers, which carry the metadata every request needs.

Key Principles:

- Every HTTP method has a semantic meaning — using the wrong one misleads both humans and systems
- API documentation always specifies the required method — read it before coding
- CRUD maps cleanly to HTTP methods; memorize this mapping

Examples:

- "Add a comment to a post" → POST `/posts/5/comments`
 - "Get a user's profile" → GET `/users/3`
 - "Change a user's username" → PATCH `/users/3`
-

02.3 HTTP Methods Challenge 📖 (~600 words)

Challenge Prompt: Given the following list of operations for a blog API, write the correct `fetch()` call for each one. Use `https://jsonplaceholder.typicode.com` as the base URL. You do not need to handle the response — just construct and send the request correctly for each case:

1. Get all posts
2. Get post number 5
3. Create a new post with title `"My Post"`, body `"Hello world"`, and `userId 1`

4. Update only the title of post number 5 to "Updated Title"
5. Delete post number 5

Solution Code:

```
// 1. Get all posts
fetch("https://jsonplaceholder.typicode.com/posts");

// 2. Get post number 5
fetch("https://jsonplaceholder.typicode.com/posts/5");

// 3. Create a new post
fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "My Post", body: "Hello world", userId: 1 })
});

// 4. Update only the title of post 5
fetch("https://jsonplaceholder.typicode.com/posts/5", {
  method: "PATCH",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Updated Title" })
});

// 5. Delete post number 5
fetch("https://jsonplaceholder.typicode.com/posts/5", {
  method: "DELETE"
});
```

Challenge Code:

```
// Challenge: Write the correct fetch() call for each operation
// Base URL: https://jsonplaceholder.typicode.com

// 1. Get all posts
// your code here

// 2. Get post number 5
// your code here

// 3. Create a new post with title "My Post", body "Hello world", userId 1
// your code here

// 4. Update only the title of post number 5 to "Updated Title"
// your code here

// 5. Delete post number 5
// your code here
```

Section 03: Building Requests

03.0 Request Headers 📖 (~500 words)

Learning Objectives:

- Explain what request headers are and why they exist
- Identify the two most important headers for API communication
- Know when and how to include headers in a `fetch()` call

Content Outline:

- Headers are metadata: they describe the request, not the data itself
- **Content-Type**: tells the server how the body is formatted (most commonly `application/json`)
- **Authorization**: proves who you are — carries the token or credentials
- Other common headers: **Accept**, **Cache-Control** (brief mentions)
- How to add headers to a `fetch()` call in JavaScript

Transitions: Headers describe the request. The next lesson covers the body — the actual data you send to the server.

Key Principles:

- Always set **Content-Type**: `application/json` when sending JSON — without it, the server may not parse the body correctly
- The **Authorization** header is how you prove identity on every single request — the server does not remember you between calls
- Headers are case-insensitive but should follow convention for readability

Examples:

- A POST request without **Content-Type** often results in a 400 Bad Request — the server cannot parse the body
 - **Authorization**: `Bearer eyJhbGci...` sends a JWT token to a protected endpoint
 - **Accept**: `application/json` tells the server you want JSON back, not HTML
-

03.1 Request Bodies 📖 (~500 words)

Learning Objectives:

- Explain what a request body is and when it is required
- Serialize a JavaScript object into a JSON string for sending
- Understand the relationship between Content-Type and body format

Content Outline:

- The body is the payload: the data you are sending to the server
- GET and DELETE requests have no body — data travels in the URL
- POST, PUT, and PATCH requests carry a body

- JSON is the most common body format: `JSON.stringify()` converts objects to strings
- URL-encoded form data: an alternative format used by HTML forms
- The body must match the `Content-Type` header — mismatches cause errors

Transitions: Most requests send JSON. The next lesson covers a special case: sending files, which requires a different format entirely.

Key Principles:

- Always `JSON.stringify()` your data before putting it in the body — raw objects cannot travel over HTTP
- The format of the body must match the `Content-Type` header exactly
- GET requests carry parameters in the URL, not the body

Examples:

- `body: JSON.stringify({ username: "ana", password: "1234" })` sends login credentials as JSON
 - An HTML form with `method="POST"` sends URL-encoded data by default, not JSON
 - Sending `body: { name: "test" }` without `JSON.stringify()` sends the string `[object Object]` — a common bug
-

03.2 Sending Files 📖 (~500 words)

Learning Objectives:

- Explain why files require a different body format than JSON
- Construct a `FormData` object to include a file in a request
- Know when NOT to use JSON for a request body

Content Outline:

- Files are binary data — JSON cannot represent them
- `FormData`: the browser API for building multipart/form-data bodies
- How to append text fields and file inputs to a `FormData` object
- Do NOT set `Content-Type` manually when using `FormData` — the browser sets the boundary automatically
- Common use cases: profile photo upload, document submission, CSV import

Transitions: You now know how to build any kind of request body. The next lesson brings everything together — method, headers, and body — into a complete assembled request.

Key Principles:

- Never try to send a file as JSON — use `FormData` and multipart/form-data
- Let the browser set `Content-Type` automatically when using `FormData` — overriding it breaks the boundary
- `FormData` can mix file inputs with regular text fields in the same request

Examples:

- Uploading a profile photo: append the File object from an `<input type="file">` to FormData and POST it
 - A document submission form: FormData with a text field for the title and a file field for the PDF
 - Setting **Content-Type: multipart/form-data** manually when using FormData causes the request to fail silently
-

03.3 Assembling a Complete Request 📖 (~500 words)

Learning Objectives:

- Combine method, headers, and body into a correctly structured `fetch()` call
- Identify what each part of the request does before sending it
- Avoid the most common request assembly mistakes

Content Outline:

- The full `fetch()` signature: URL, method, headers, body
- Decision tree: which method? does it need a body? which Content-Type?
- Reading an API specification and translating it into a `fetch()` call
- Anti-pattern: sending a body with GET
- Anti-pattern: forgetting to stringify the body
- Anti-pattern: setting Content-Type manually when using FormData

Transitions: You can now build a complete, correctly structured request. The next lesson is a code challenge where you will assemble requests from scratch given a set of specifications.

Key Principles:

- Before writing a single line of code, identify: method, URL, headers needed, body format
- A well-assembled request follows the API specification exactly — no guessing
- The three most common assembly bugs are: wrong method, missing Content-Type, and unstringified body

Examples:

- Spec says: "POST /comments, body: { postId, text }, auth required" → method POST, Content-Type application/json, Authorization Bearer, body `JSON.stringify({postId, text})`
 - Spec says: "DELETE /posts/:id, auth required" → method DELETE, Authorization Bearer, no body
 - Spec says: "Upload avatar: POST /users/:id/avatar, file upload" → method POST, FormData body, no manual Content-Type
-

03.4 Request Builder Challenge 📖 (~600 words)

Challenge Prompt: You are given four API specifications below. Write a correctly assembled `fetch()` call for each one. Pay attention to method, headers, body format, and whether authentication is required. Use `"YOUR_TOKEN_HERE"` as a placeholder for the Bearer token where needed.

1. `GET /products` — no auth required, no body
2. `POST /products` — auth required, body: `{ name: "Laptop", price: 999 }`

3. **PATCH** `/products/7` — auth required, body: `{ price: 849 }`
4. **POST** `/products/7/image` — auth required, body: a file from a variable called `imageFile`

Solution Code:

```
// 1. GET /products
fetch("https://api.example.com/products");

// 2. POST /products
fetch("https://api.example.com/products", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer YOUR_TOKEN_HERE"
  },
  body: JSON.stringify({ name: "Laptop", price: 999 })
});

// 3. PATCH /products/7
fetch("https://api.example.com/products/7", {
  method: "PATCH",
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer YOUR_TOKEN_HERE"
  },
  body: JSON.stringify({ price: 849 })
});

// 4. POST /products/7/image
const formData = new FormData();
formData.append("image", imageFile);

fetch("https://api.example.com/products/7/image", {
  method: "POST",
  headers: {
    "Authorization": "Bearer YOUR_TOKEN_HERE"
  },
  body: formData
});
```

Challenge Code:

```
// Challenge: Assemble a correct fetch() call for each specification
// Use "YOUR_TOKEN_HERE" as the Bearer token placeholder where auth is required

// 1. GET /products — no auth, no body
// your code here

// 2. POST /products — auth required, body: { name: "Laptop", price: 999 }
// your code here
```

```
// 3. PATCH /products/7 – auth required, body: { price: 849 }  
// your code here  
  
// 4. POST /products/7/image – auth required, body: file stored in variable  
imageFile  
// your code here
```

Section 04: Reading Responses

04.0 HTTP Status Codes 📖 (~500 words)

Learning Objectives:

- Interpret the meaning of the most common HTTP status codes
- Distinguish between client errors and server errors
- Use status codes to decide how to handle a response in code

Content Outline:

- Status codes are three-digit numbers that summarize the outcome of a request
- The five families: 1xx informational, 2xx success, 3xx redirection, 4xx client error, 5xx server error
- Must-know codes: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Unprocessable Entity, 500 Internal Server Error
- A 404 is not a crash — it is a valid response that means "not found"
- `fetch()` does not throw on 4xx or 5xx — you must check `response.ok` or `response.status` manually

Transitions: You now know how to read the status code. The next lesson covers the response body — the actual data the server sends back.

Key Principles:

- A response with a 4xx status is your fault (bad request); a 5xx is the server's fault
- `fetch()` only rejects its promise on network failure — a 404 or 500 still resolves
- Always check the status code before trying to parse the body

Examples:

- A successful POST to create a user returns 201 Created with the new user object in the body
- Sending a request to a protected endpoint without a token returns 401 Unauthorized
- Trying to access a resource that does not exist returns 404 Not Found — handle it gracefully, do not crash

04.1 Reading the Response Body 📖 (~500 words)

Learning Objectives:

- Parse a JSON response body using `response.json()`
- Understand why `response.json()` returns a promise

- Handle empty responses and non-JSON responses correctly

Content Outline:

- The response body is a stream — it must be read asynchronously
- `response.json()`: parses the body as JSON, returns a promise
- `response.text()`: reads the body as plain text
- `response.blob()`: reads binary data (files, images)
- What happens when you call `response.json()` on an empty body (204 responses)
- Checking `response.ok` before parsing to avoid parsing error HTML into a broken object

Transitions: You can now read both the status and the body of a response. The next lesson is a challenge where you will interpret real API responses and handle different outcomes correctly.

Key Principles:

- Always check `response.ok` or `response.status` before calling `response.json()` — parsing an error response as JSON can mislead you
- A 204 No Content response has no body — calling `response.json()` on it throws an error
- The body can only be read once — if you read it with `response.text()`, you cannot then read it with `response.json()`

Examples:

- A successful GET returns 200 with a JSON body → check `response.ok`, then call `response.json()`
- A DELETE returns 204 with no body → check status, skip body parsing
- A 500 error sometimes returns an HTML error page → `response.json()` throws, wrapping it in try/catch saves the app

04.2 Response Interpreter Challenge 🖥️ (~600 words)

Challenge Prompt: Complete the `handleResponse` function below. It receives a `fetch()` response object and must:

1. If the status is 200, parse and return the JSON body
2. If the status is 201, parse and return the JSON body
3. If the status is 204, return `null`
4. If the status is 404, return the string `"Not found"`
5. If the status is 401, return the string `"Unauthorized"`
6. For any other status, return the string `"Unexpected error"`

Solution Code:

```
async function handleResponse(response) {
  if (response.status === 200 || response.status === 201) {
    return await response.json();
  }
  if (response.status === 204) {
    return null;
  }
  if (response.status === 404) {
    return "Not found";
  }
  if (response.status === 401) {
    return "Unauthorized";
  }
  return "Unexpected error";
}
```

```
}  
if (response.status === 404) {  
  return "Not found";  
}  
if (response.status === 401) {  
  return "Unauthorized";  
}  
return "Unexpected error";  
}
```

Challenge Code:

```
async function handleResponse(response) {  
  // If status is 200 or 201: parse and return the JSON body  
  // your code here  
  
  // If status is 204: return null  
  // your code here  
  
  // If status is 404: return "Not found"  
  // your code here  
  
  // If status is 401: return "Unauthorized"  
  // your code here  
  
  // For anything else: return "Unexpected error"  
  // your code here  
}
```

Section 05: Tools and Authentication

05.0 Basic Authentication (~500 words)

Learning Objectives:

- Explain what Basic Authentication is and how it works
- Construct a Basic Auth header manually
- Understand why Basic Auth is only safe over HTTPS

Content Outline:

- Basic Auth sends username and password encoded in Base64 on every request
- The header format: `Authorization: Basic <base64(username:password)>`
- Base64 is encoding, not encryption — it is trivially reversible
- Why Basic Auth is safe only over HTTPS: without TLS, the credentials are exposed
- When Basic Auth is appropriate: internal tools, simple scripts, initial token exchange
- Basic Auth vs token-based auth: use Basic Auth once to get a token, then use the token

Transitions: Basic Auth is simple but limited. The next lesson covers a critical anti-pattern — assuming the server remembers who you are between requests.

Key Principles:

- Base64 is not security — it is just encoding. Never treat a Basic Auth header as encrypted
- Always use HTTPS when sending any form of credentials
- Basic Auth is best used as a one-time exchange to receive a more durable token

Examples:

- **Authorization: Basic** `dXNlcjpwYXNz` — the string `user:pass` encoded in Base64
 - A CLI script authenticating to an internal admin API using Basic Auth over HTTPS
 - Using Basic Auth to hit `POST /auth/token` once and store the returned JWT for all subsequent requests
-

05.1 Anti-Pattern: The Forgotten Header 📖 (~500 words)

Learning Objectives:

- Recognize the two most common header-related mistakes in API communication
- Understand the consequences of each mistake
- Apply a simple checklist to avoid them

Content Outline:

- Anti-pattern 1 — Missing Content-Type: sending a JSON body without **Content-Type: application/json** causes the server to reject or misparse the body, resulting in a 400 error
- Anti-pattern 2 — Missing Authorization: making a request to a protected endpoint without the **Authorization** header results in a 401 Unauthorized — the server does not remember you from the last request
- Why these mistakes are so common: they are invisible in the request body and only surface as cryptic server errors
- How to avoid them: build a mental checklist before every `fetch()` call
 - Does this endpoint require auth? → add Authorization header
 - Am I sending a JSON body? → add Content-Type header
 - Am I sending a file? → use `FormData`, do NOT set Content-Type manually

Transitions: You now know what to watch for. The next lesson introduces Postman — the tool that makes these mistakes visible before they reach your code.

Key Principles:

- The server has no memory — if your token was valid yesterday, it still needs to travel in today's request
- A missing Content-Type is the most common cause of mysterious 400 Bad Request errors
- Building a pre-send checklist is a professional habit, not a beginner crutch

Examples:

- A POST request with a JSON body but no Content-Type header → server returns 400, body is treated as plain text
 - A GET request to `/me` without an Authorization header → server returns 401, even though the user logged in moments ago in a different request
 - A FormData request with a manually set Content-Type → multipart boundary is broken, file upload silently fails
-

05.2 Using Postman (~500 words)

Learning Objectives:

- Send a request using Postman without writing any code
- Inspect the full request and response in Postman's interface
- Use Postman to verify API behavior before integrating into a frontend

Content Outline:

- What Postman is: a GUI client for sending HTTP requests and inspecting responses
- Why use Postman: test an API independently of your frontend code — isolate problems faster
- Creating a request: setting the method, URL, headers, and body in the interface
- Reading the response: status code, response body, response headers, response time
- Saving requests into a collection for reuse
- The professional workflow: test in Postman first, code second

Transitions: You have now seen all the tools. The next section is your assessment — a full API workflow challenge followed by a knowledge check.

Key Principles:

- Postman separates API testing from frontend development — if it works in Postman but not in your code, the bug is in your code
- Always test an API endpoint in Postman before writing the `fetch()` call — it saves debugging time
- Collections keep your API tests organized and shareable with teammates

Examples:

- Setting **Authorization: Bearer <token>** in the Headers tab and sending a request to a protected endpoint
 - Pasting a JSON object into the Body tab (raw, JSON) and verifying the server creates the resource correctly
 - Seeing a 422 response in Postman tells you the request is malformed before you write a single line of frontend code
-

05.3 Postman Challenge (~600 words)

Challenge Prompt: Using Postman (not code), complete the following four tasks against the JSONPlaceholder API (<https://jsonplaceholder.typicode.com>). For each task, take note of the status code and the response body you receive.

1. Send a GET request to `/posts/1` and confirm the response status is 200
2. Send a POST request to `/posts` with a JSON body containing `{ "title": "Test", "body": "Hello", "userId": 1 }` — set the Content-Type header to `application/json` and confirm the response status is 201
3. Send a PATCH request to `/posts/1` with body `{ "title": "Updated" }` — confirm the status is 200
4. Send a DELETE request to `/posts/1` — confirm the status is 200

Write your observations as comments in the file below: the status code you received and one sentence describing what the response body contained (or did not contain).

Solution Code:

```
// Task 1 – GET /posts/1
// Status: 200
// Response body: a single post object with id, title, body, and userId fields

// Task 2 – POST /posts
// Status: 201
// Response body: the newly created post object with an id assigned by the server

// Task 3 – PATCH /posts/1
// Status: 200
// Response body: the post object with the title updated to "Updated"

// Task 4 – DELETE /posts/1
// Status: 200
// Response body: an empty object {}
```

Challenge Code:

```
// Record your Postman observations here

// Task 1 – GET /posts/1
// Status:
// Response body:

// Task 2 – POST /posts with { "title": "Test", "body": "Hello", "userId": 1 }
// Status:
// Response body:

// Task 3 – PATCH /posts/1 with { "title": "Updated" }
// Status:
// Response body:

// Task 4 – DELETE /posts/1
// Status:
// Response body:
```

Section 06: Assessment

06.0 Full API Workflow Challenge 🖨️ (~700 words)

Challenge Prompt: Complete the four functions below. Each one performs a different operation against the JSONPlaceholder API. All functions must handle the response correctly: check the status, parse the body where applicable, and return the result. Use `async/await`.

1. `getPost(id)` — fetch a single post by ID, return the post object, return `null` if not found
2. `createPost(title, body, userId)` — create a new post, return the created post object
3. `updatePostTitle(id, newTitle)` — update only the title of a post, return the updated post object
4. `deletePost(id)` — delete a post, return `true` if successful, `false` otherwise

Solution Code:

```
const BASE_URL = "https://jsonplaceholder.typicode.com";

async function getPost(id) {
  const response = await fetch(`${BASE_URL}/posts/${id}`);
  if (response.status === 404) return null;
  return await response.json();
}

async function createPost(title, body, userId) {
  const response = await fetch(`${BASE_URL}/posts`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title, body, userId })
  });
  return await response.json();
}

async function updatePostTitle(id, newTitle) {
  const response = await fetch(`${BASE_URL}/posts/${id}`, {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title: newTitle })
  });
  return await response.json();
}

async function deletePost(id) {
  const response = await fetch(`${BASE_URL}/posts/${id}`, {
    method: "DELETE"
  });
  return response.ok;
}
```

Challenge Code:

```
const BASE_URL = "https://jsonplaceholder.typicode.com";

// 1. Fetch a single post by ID
// Return the post object, or null if not found (404)
async function getPost(id) {
  // your code here
}

// 2. Create a new post
// Return the created post object
async function createPost(title, body, userId) {
  // your code here
}

// 3. Update only the title of a post
// Return the updated post object
async function updatePostTitle(id, newTitle) {
  // your code here
}

// 4. Delete a post
// Return true if successful, false otherwise
async function deletePost(id) {
  // your code here
}
```

06.1 Knowledge Check 🧠 (~700 words)

Learning Objectives:

- Verify understanding of HTTP methods, request structure, response handling, and authentication
- Identify correct and incorrect API usage patterns
- Apply course concepts to realistic scenarios

Question Format: Multiple choice

Questions:

Q1. You need to update only the `email` field of a user record without changing any other fields. Which HTTP method should you use?

- A) PUT
- B) POST
- C) PATCH ☒
- D) GET

Explanation: PATCH updates only the fields you send. PUT replaces the entire resource.

Q2. You send a POST request with a JSON body but receive a 400 Bad Request response. The most likely cause is:

- A) The server is down
- B) The Authorization header is missing
- C) The Content-Type header is missing or incorrect ☒
- D) GET should have been used instead of POST

Explanation: Without Content-Type: application/json, the server cannot parse the body correctly and rejects the request.

Q3. You call `fetch("/api/users/99")` and the user does not exist. What does `fetch()` do?

- A) Throws an error automatically
- B) Resolves the promise with a response object where status is 404 ☒
- C) Returns null
- D) Rejects the promise

Explanation: `fetch()` only rejects on network failure. A 404 is a valid HTTP response — you must check the status yourself.

Q4. Which of the following correctly sends a Bearer token in a `fetch()` request?

- A) `body: { token: "abc123" }`
- B) `headers: { "Token": "abc123" }`
- C) `headers: { "Authorization": "Bearer abc123" }` ☒
- D) `params: { authorization: "Bearer abc123" }`

Explanation: The Authorization header with the Bearer prefix is the standard way to send a JWT token.

Q5. You are uploading a file using `FormData`. Which of the following is correct?

- A) Set `Content-Type: multipart/form-data` manually in the headers
- B) Set `Content-Type: application/json` in the headers
- C) Do not set Content-Type — let the browser set it automatically ☒
- D) Stringify the `FormData` object before placing it in the body

Explanation: When using `FormData`, the browser automatically sets Content-Type including the required boundary parameter. Setting it manually breaks the upload.

Q6. A DELETE request to `/posts/5` returns status 204. What should your code do next?

- A) Call `response.json()` to read the confirmation
- B) Treat it as an error — 204 is not a success code
- C) Skip body parsing — 204 means success with no content ☒
- D) Retry the request — no body means the deletion failed

Explanation: 204 No Content is a success status. Calling `response.json()` on a 204 will throw an error because there is no body to parse.

Evaluation Criteria:

- 6/6: Excellent — full command of request structure and response handling
 - 4-5/6: Good — minor gaps in header behavior or response parsing
 - 2-3/6: Needs review — revisit sections on HTTP methods and request assembly
 - 0-1/6: Restart recommended — return to Section 01 and work through again
-

Section 07: Conclusion

07.0 What Comes Next (~450 words)

Content Outline:

- Course recap: you can now construct any HTTP request and interpret any response
- What you practiced: the full request-response cycle, all five HTTP methods, headers and bodies, status codes, Postman, and Basic Authentication
- Connection to the bigger picture: every modern web application is built on top of exactly what you learned here — APIs are not a special feature, they are the infrastructure
- What comes next: in the following lessons you will build your own API with FastAPI, and everything you learned here about requests and responses will apply in reverse — you will be the one writing the endpoints that others call
- A note on Postman: keep it open as you build your backend — test your own endpoints the same way you tested external ones
- Encouragement: you have crossed a significant threshold — you can now have a conversation between a frontend and a backend, and that is the foundation of every full-stack application you will ever build

Note: This is conclusion only — no theory, exercises, or assessment.